

NAPCO Nucleus — Sandbox Setup

From git clone to your first requirement draft, channel by channel

1. Prerequisites

Before cloning, make sure your machine has:

- **Windows 10 or 11.** Teams chat ingest reads the local IndexedDB cache, which is Windows-only.
- **Python 3.11 or newer.** Confirm with `python --version`.
- **Git for Windows.**
- **Microsoft Teams desktop**, signed in to your work account, with the chats you'll be pulling already open.
- **Microsoft Outlook** (or another IMAP-aware mail client) configured for the same Gmail account you'll authenticate below. Drafts will appear here for manual send.
- **A Gmail account** — used as the IMAP poller and the From identity on outgoing drafts.
- **~5 GB free disk space** for the Whisper model + workspace.

2. Clone the repo and install dependencies

Open PowerShell or Git Bash and run:

```
git clone https://github.com/napco-labs/napco-nucleus.git
cd napco-nucleus
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

3 to 5 minutes. The heaviest deps are faster-whisper and onnxruntime (used for audio transcription); everything else is small.

Create your `.env` file in the project root. Start empty — the sections below tell you which variables to add as you set up each channel:

```
type nul > .env (PowerShell)
touch .env (Git Bash)
```

3. Channel 1 — Email

Used for both pulling email content and pushing email drafts into your Drafts folder. Authentication is via Gmail App Password — your normal Google password will not work, because Google blocks IMAP login with a regular password when 2FA is on.

3.1 Generate a Gmail App Password

- 1 Open <https://myaccount.google.com/security>.
- 2 Confirm 2-Step Verification is ON. (If not, turn it on first — App Passwords are gated on 2SV.)
- 3 Click **App passwords**. (Search if you don't see it.)
- 4 Pick **Mail** as the app, give it a label like "NAPCO Nucleus", click **Create**.

NAPCO Nucleus — Sandbox Setup

From git clone to your first requirement draft, channel by channel

- 5 Copy the 16-character password Google shows you. You won't see it again.

3.2 Add to .env

```
REQ_IMAP_HOST=imap.gmail.com
REQ_IMAP_PORT=993
REQ_IMAP_USER=your.address@gmail.com
REQ_IMAP_PASSWORD=<the 16-char app password, no spaces>
SMTP_FROM=your.address@gmail.com
SMTP_FROM_NAME=Your Name
VERIFICATION_TO=client@example.com
```

Smoke test. Make sure IMAP login works:

```
python -m mail.pull_email --last-minutes 5
```

Should print "Pulling emails from mailbox..." and either match 0 or N emails. "IMAP login failed" means the App Password is wrong; regenerate.

4. Channel 2 — Google Drive

The Drive ingester reads files from one specified folder using a Google service account. The service account needs read access to that folder; you grant it by sharing the folder with the service account's email.

4.1 Create a service account + JSON key

- 1 Open <https://console.cloud.google.com/>.
- 2 Select or create a project (any project — this is a free identity).
- 3 Navigate **IAM & Admin** → **Service Accounts** → **Create Service Account**. Give it a name like "napco-nucleus-drive".
- 4 Skip the role-grant step (no project-level role needed).
- 5 On the new account's **Keys** tab, click **Add Key** → **Create new key** → **JSON**. A JSON file downloads.
- 6 Note the `client_email` field inside the JSON (looks like `name@project.iam.gserviceaccount.com`).
- 7 Save the JSON somewhere outside the repo — for example `C:\Users\<you>\google-credentials.json`.

4.2 Enable the Drive API on the project

Cloud Console → **APIs & Services** → **Enable APIs and Services** → search "Google Drive API" → **Enable**. Same-project as the service account.

4.3 Share your Drive folder with the service account

- 1 Create or pick a folder in Google Drive that will hold your incoming files.
- 2 Right-click → Share. Paste the service account's `client_email`. Permission: **Viewer** is enough; **Editor** is fine if you ever want write access.

NAPCO Nucleus — Sandbox Setup

From git clone to your first requirement draft, channel by channel

- 3 Open the folder; copy the ID from the URL:
`drive.google.com/drive/folders/<THIS_PART>`.

4.4 Add to .env

```
GOOGLE_CREDENTIALS_PATH=C:\Users\<you>\google-credentials.json
GDRIVE_AUDIO_FOLDER_ID=<the folder ID>
# Required only if you'll have audio in the folder:
GROQ_API_KEY=<groq key from console.groq.com>
```

Smoke test. List one file from the folder:

```
python -m drive.pull_drive --last-files 1
```

If the folder is empty, prints "Matched 0 of 0". If the service account isn't shared, you'll get a 404 from Google.

5. Channel 3 — Teams chat

No API or token. The reader walks the local IndexedDB that Teams desktop populates when it's signed in. So the only setup is having the right chats already loaded in your Teams desktop and adding name aliases for any chat with a blank title.

5.1 Pre-flight

- Open the Teams desktop app and sign in.
- Navigate into each chat or group you intend to pull from at least once. Teams only fully syncs a chat's messages to the local cache after it has been opened.
- Optional: leave Teams running. The reader works fine while Teams is open (no file lock).

5.2 First-time chat discovery

From the project root:

```
python -m teams.list_chats
```

Walks the IndexedDB and writes `data/teams/chats.txt` listing every chat with its registry number, `conversation_id`, and participants. Note the chat numbers of the groups you'll pull from.

5.3 CHAT_ALIASES — add names you can type

Many group chats — especially older legacy threads — have empty titles in the cache. To call them by name, add aliases to `.env`. Format is comma-separated `alias=identifier` where identifier is a chat number or a full `conversation_id`.

Example for a group called Acme (chat #45) with three members:

```
CHAT_ALIASES=Acme=45,Alice=45,Bob=45,Charlie=45
```

Each member name resolves to the same group chat. You can also narrow to one person's messages with `--sender Alice` when pulling.

5.4 Smoke test

NAPCO Nucleus — Sandbox Setup

From git clone to your first requirement draft, channel by channel

```
python -m teams.pull_chat --name Acme --last-minutes 60
```

Should resolve the alias and print "Messages in window: N". If you get "No chat matching ...", either the alias is missing from .env or the chat hasn't been opened in Teams desktop yet.

6. Channel 4 — Teams audio call

Records two WAV tracks — your microphone and the system speaker loopback — so the transcript distinguishes "You" from "Other". Transcription runs locally with faster-whisper large-v3 (Bangla → English). No cloud API needed.

6.1 Pre-flight

- Confirm your default microphone works (any voice app you normally use).
- Confirm your default speakers / headset are configured. The recorder uses Windows WASAPI loopback to capture whatever is playing — so the call audio must come through the same speakers.
- No driver install needed: PyAudioWPatch ships its own native binary.

6.2 One-time Whisper download (~3 GB)

First run downloads the model to C:\Users\<you>\.cache\huggingface\hub. Pre-cache it so your first real meeting doesn't wait for the download:

```
python -c "from faster_whisper import WhisperModel; WhisperModel('large-v3', device='cpu', compute_type='int8')"
```

Subsequent loads are 3-5 seconds.

6.3 Smoke test (record + transcribe a 10-second clip)

- 1 Run `python -m teams.record_call` in one terminal.
- 2 Talk for ~10 seconds. Play any audio through your speakers if you also want a 'speaker' track.
- 3 In a second terminal: `python -m teams.stop_recording`. The recorder closes the WAVs and exits.
- 4 Transcribe and append: `python pull_meeting.py`. Confirms speaker labels (You / Other) in the output.

7. First end-to-end run

With all channels set up, do one full pass to see the whole pipeline producing a real client email draft:

- 1 Have a 1-minute Teams call with someone, talk through one or two requirements.
- 2 Send yourself a test email with a simple requirement-flavored ask.
- 3 Drop a small PDF or .txt file with another requirement into the Drive folder.
- 4 Stop the recording (`python -m teams.stop_recording`).
- 5 Pull each channel:
`python -m teams.pull_chat --name <group> --last-minutes 30`

NAPCO Nucleus — Sandbox Setup

From git clone to your first requirement draft, channel by channel

```
python -m mail.pull_email --last-minutes 30
python -m drive.pull_drive --last-files 1
python pull_meeting.py
```

- 6 Identify and draft: `python agent.py --task verify_session`.
- 7 Open Outlook → Drafts. The verification email + attached Word doc should be there. Send it (or don't — it's still your call).

8. You're set up

Open the User Manual for day-to-day reference: per-channel command options, time-window flags, sender filters, and troubleshooting. When you upgrade the project, run `git pull` and then `pip install -r requirements.txt` to pick up any new dependencies.